A02 on Fitting and Alignment

Student Details

Field	Details
Name	D.M.G.H. Adikaram
Index Number	230018T
Github	https://github.com/Gimadikaram/A02-on-Fitting-and-Alignment.git

Question 1: Sunflower Detection Using Laplacian of Gaussian

The objective is to detect circular sunflower blobs at different scales.

The image is converted to grayscale, and the scale-normalized Laplacian of Gaussian (LoG) is calculated for multiple values of σ:

$R(x, y, \sigma) = \sigma^2 \nabla^2 (G\sigma * I)(x, y)$

Dark blobs surrounded by brighter regions produce positive LoG responses. Therefore, local maxima are detected across both spatial coordinates and scale.

For an ideal uniform circular blob, its radius is approximately:

$r = \sqrt{2}\,\sigma$

In [5]:
```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt
from scipy.ndimage import maximum_filter

image_path = "C:/Users/ASUS/Downloads/CV ASSIGNMENT/A02-on-Fitting-and-
Alignment/a2images/the_berry_farms_sunflower_field.jpeg"

im = cv.imread(image_path, cv.IMREAD_REDUCED_COLOR_4)
if im is None:
    raise FileNotFoundError(image_path)

rgb = cv.cvtColor(im, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY).astype(np.float32) / 255.0
h, w = gray.shape

# Scale-normalized LoG
sigmas = np.linspace(1, 10, 10)
threshold = 0.25
scale_space = np.empty((h, w, len(sigmas)), dtype=np.float32)

for i, sigma in enumerate(sigmas):
    half_width = int(np.ceil(3 * sigma))
    coordinates = np.arange(-half_width, half_width + 1)
    X, Y = np.meshgrid(coordinates, coordinates)
    r_squared = X**2 + Y**2
```

```
    kernel = (
        1 / (np.pi * sigma**2)
        * (r_squared / (2 * sigma**2) - 1)
        * np.exp(-r_squared / (2 * sigma**2))
    )
    kernel -= kernel.mean()

    scale_space[:, :, i] = cv.filter2D(
        gray, cv.CV_32F, kernel, borderType=cv.BORDER_REFLECT101
    )

# Local maxima across position and scale
mask = (
    (scale_space == maximum_filter(scale_space, size=(3, 3, 3)))
    & (scale_space > threshold)
)
mask[:, :, 0] = False
mask[:, :, -1] = False

for i, sigma in enumerate(sigmas):
    m = int(np.ceil(np.sqrt(2) * sigma))
    mask[:m, :, i] = mask[-m:, :, i] = False
    mask[:, :m, i] = mask[:, -m:, i] = False

ys, xs, ss = np.where(mask)
radii = np.sqrt(2) * sigmas[ss]
responses = scale_space[ys, xs, ss]

# Five largest non-overlapping circles
chosen = []

for idx in np.lexsort((-responses, -radii)):
    x, y, r = xs[idx], ys[idx], radii[idx]

    if all(
        np.hypot(x - cx, y - cy) > r + cr
        for cx, cy, cr, _ in chosen
    ):
        chosen.append((x, y, r, sigmas[ss[idx]]))

    if len(chosen) == 5:
        break

# Display results
fig, axes = plt.subplots(1, 2, figsize=(14, 7))

for ax in axes:
    ax.imshow(rgb)
    ax.axis("off")

for x, y, r in zip(xs, ys, radii):
```

```
    axes[0].add_patch(
        plt.Circle((x, y), r, color="red", fill=False, linewidth=1)
    )

for number, (x, y, r, sigma) in enumerate(chosen, 1):
    axes[1].add_patch(
        plt.Circle((x, y), r, color="cyan", fill=False, linewidth=1.5)
    )
    axes[1].text(x + r + 2, y, str(number), color="white", fontsize=12)

axes[0].set_title(f"Detected blobs: {len(xs)}")
axes[1].set_title("Largest non-overlapping blobs")
plt.tight_layout()
plt.show()

# Report parameters in reduced-image pixels
print(f"Sigma range: {sigmas.min():.1f}-{sigmas.max():.1f} pixels")
print(f"Response threshold: {threshold}")

for number, (x, y, r, sigma) in enumerate(chosen, 1):
    print(
        f"{number}: centre=({x}, {y}), "
        f"radius={r:.2f} pixels, sigma={sigma:.1f}"
    )
```




```
Sigma range: 1.0-10.0 pixels
Response threshold: 0.25
1: centre=(282, 338), radius=9.90 pixels, sigma=7.0
2: centre=(106, 255), radius=8.49 pixels, sigma=6.0
3: centre=(178, 261), radius=8.49 pixels, sigma=6.0
4: centre=(179, 327), radius=8.49 pixels, sigma=6.0
5: centre=(53, 174), radius=8.49 pixels, sigma=6.0
```

Blob Detection Using Scale-Normalized LoG

The image was loaded at one-quarter resolution, converted to grayscale, and normalized to $[0, 1]$. Scale-normalized Laplacian of Gaussian (LoG) filters were applied at ten scales, $\sigma = 1, 2, \ldots, 10$:

$$R(x, y, \sigma) = \sigma^2 \nabla^2 (G_\sigma * I)$$

The factor σ^2 makes responses comparable across scales. Positive responses identify dark sunflower centres surrounded by brighter petals.

Blob candidates were detected as local maxima within a $3 \times 3 \times 3$ neighbourhood across position and scale. A threshold of **0.25** suppressed weak responses. This was a chosen parameter, not an optimized value. Endpoint scales and circles extending beyond the image were excluded.

Each blob was assigned a centre (x, y) and an approximate radius:

$$r = \sqrt{2}\sigma$$

All candidates were displayed, and the **five largest non-overlapping circles** were selected for reporting. Coordinates and radii refer to the reduced image.

The circles represent the **dark sunflower centres**, rather than the full petal outlines. Some false detections may remain because LoG also responds to other dark regions.

Question 2: Robust Line and Circle Fitting Using RANSAC

In [18]:

```
from scipy.optimize import minimize

np.random.seed(0)
rng = np.random.default_rng(1)

N = 100
half_n = N // 2

# Circle points
r_gt = 10
center_gt = np.array([2.0, 3.0])

t = np.random.uniform(0, 2 * np.pi, half_n)
noise = (r_gt / 16) * np.random.randn(half_n)

X_circ = center_gt + np.column_stack((
    (r_gt + noise) * np.cos(t),
    (r_gt + noise) * np.sin(t)
))

# Line points
x = np.linspace(-12, 12, half_n)
y = -x + 2 + np.random.randn(half_n)
X_line = np.column_stack((x, y))

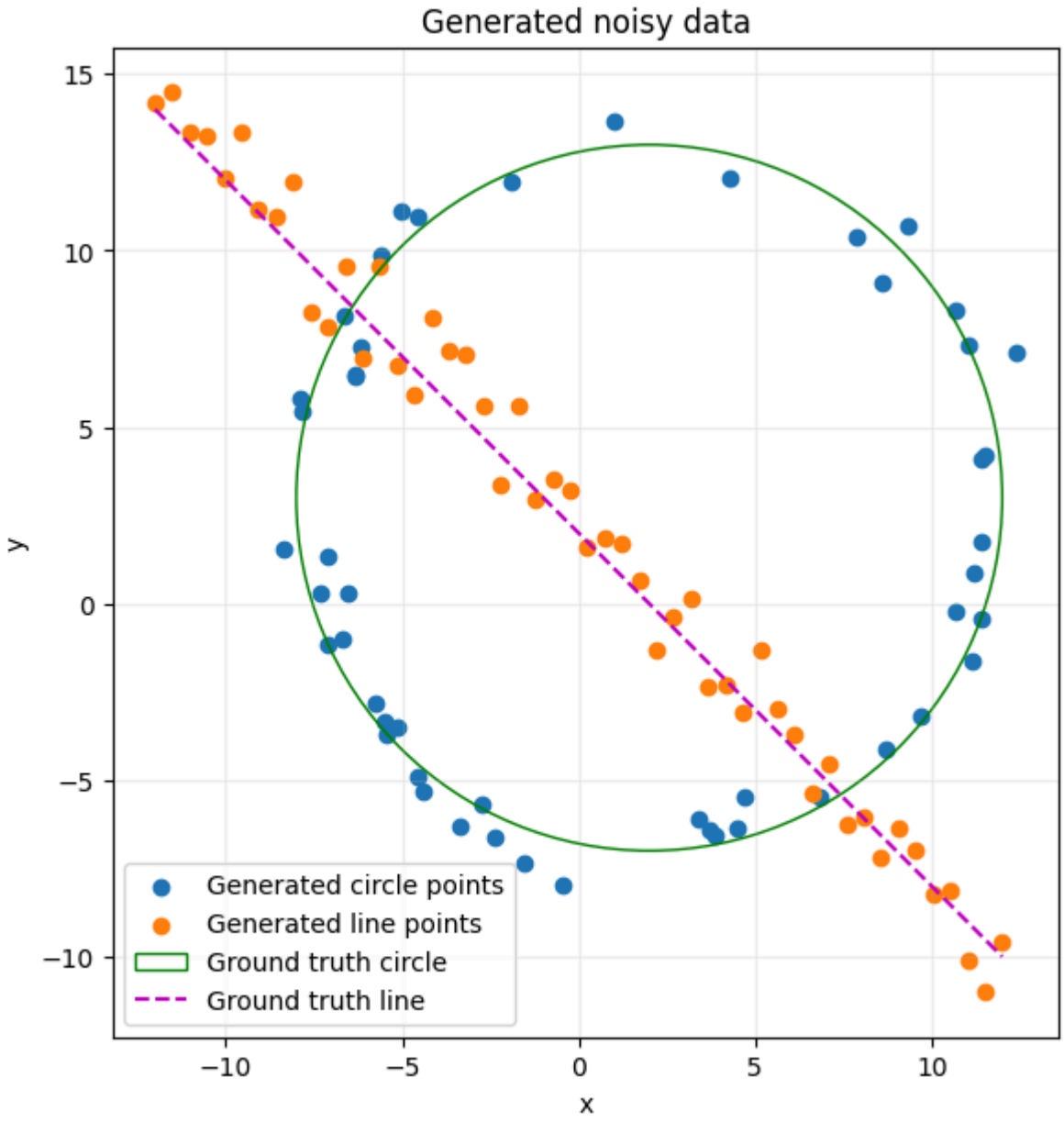
X = np.vstack((X_circ, X_line))
```

```
fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(*X_circ.T, label="Generated circle points")
ax.scatter(*X_line.T, label="Generated line points")

ax.add_patch(
    plt.Circle(center_gt, r_gt, fill=False,
               color="green", label="Ground truth circle")
)
ax.plot(x, -x + 2, "m--", label="Ground truth line")

ax.set_aspect("equal")
ax.set_title("Generated noisy data")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
ax.grid(alpha=0.2)
plt.show()

print(f"Total points: {len(X)}")
```



```
Total points: 100
```

In [19]:

```python
def fit_line(points):
    center = points.mean(axis=0)
    _, _, vt = np.linalg.svd(points - center, full_matrices=False)

    normal = vt[-1]
    normal /= np.linalg.norm(normal)
    d = normal @ center

    if d < 0:
        normal = -normal
        d = -d

    return np.array([normal[0], normal[1], d])


def line_error(model, points):
    a, b, d = model
    return np.abs(points @ np.array([a, b]) - d)


# Handles vertical lines as well.
def draw_line(ax, model, color, style, label):
    a, b, d = model
    anchor = d * np.array([a, b])
    direction = np.array([-b, a])

    extent = 3 * np.max(np.abs(X)) + abs(d)
    points = anchor + np.array([-extent, extent])[:, None] * direction

    ax.plot(*points.T, color=color, linestyle=style, label=label)


def format_axes(ax, title):
    ax.set_xlim(X[:, 0].min() - 2, X[:, 0].max() + 2)
    ax.set_ylim(X[:, 1].min() - 2, X[:, 1].max() + 2)
    ax.set_aspect("equal")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title(title)
    ax.grid(alpha=0.2)


line_threshold = 2.0
iterations = 1000
min_line_consensus = 35

best_score = (-1, -np.inf)
line_sample_model = None

for _ in range(iterations):
```

```
    sample = X[rng.choice(len(X), 2, replace=False)]

    if np.linalg.norm(sample[0] - sample[1]) < 1e-10:
        continue

    model = fit_line(sample)
    errors = line_error(model, X)
    mask = errors < line_threshold

    score = (int(mask.sum()), -float(np.sum(errors[mask]**2)))

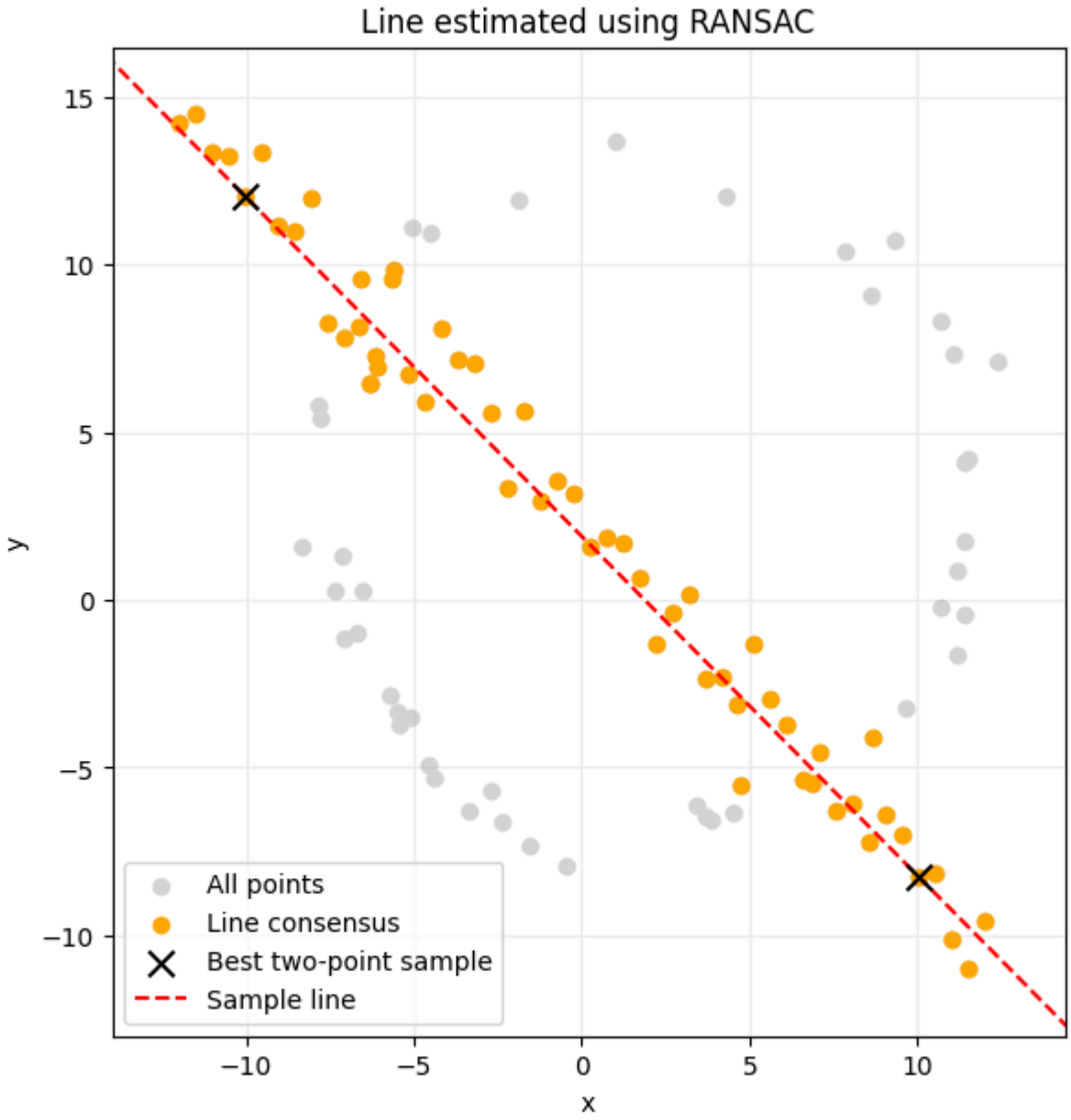
    if score > best_score:
        best_score = score
        line_sample_model = model.copy()
        line_sample_points = sample.copy()
        line_sample_mask = mask.copy()

if line_sample_model is None or best_score[0] < min_line_consensus:
    raise RuntimeError("No line met the minimum consensus requirement.")

fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(*X.T, color="lightgray", label="All points")
ax.scatter(*X[line_sample_mask].T, color="orange", label="Line consensus")
ax.scatter(
    *line_sample_points.T, color="black", marker="x", s=100,
    label="Best two-point sample", zorder=5
)

draw_line(ax, line_sample_model, "red", "--", "Sample line")
format_axes(ax, "Line estimated using RANSAC")
ax.legend()
plt.show()

a, b, d = line_sample_model
print(f"Sample line: {a:.4f}x + {b:.4f}y = {d:.4f}")
print(f"Consensus size: {line_sample_mask.sum()}")
```




```
Sample line: 0.7109x + 0.7033y = 1.3370
Consensus size: 58
```

In [20]:

```
# Total least-squares fit using the winning consensus.
line_best = fit_line(X[line_sample_mask])

# Reclassify points using the refined line.
line_mask = line_error(line_best, X) < line_threshold

if line_mask.sum() < min_line_consensus:
    raise RuntimeError("Refined line has insufficient consensus.")

remaining = X[~line_mask]

fig, axes = plt.subplots(1, 2, figsize=(13, 6))

axes[0].scatter(*X.T, color="lightgray", label="All points")
axes[0].scatter(*X[line_mask].T, color="orange", label="Final line inliers")

draw_line(axes[0], line_sample_model, "black", ":", "Sample line")
draw_line(axes[0], line_best, "red", "-", "Refined line")

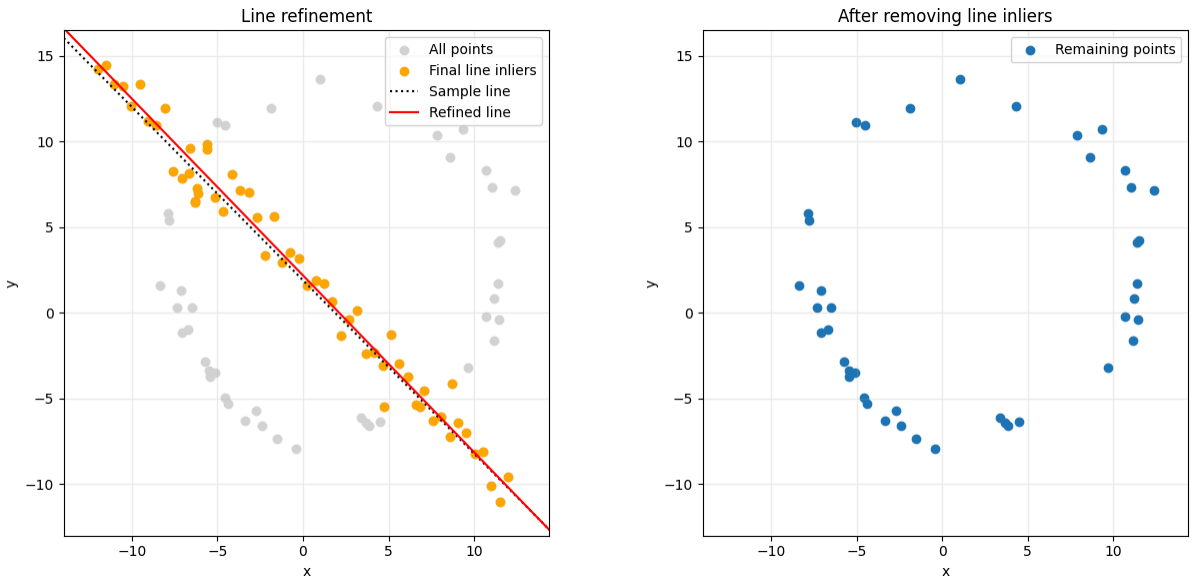
axes[1].scatter(*remaining.T, color="tab:blue", label="Remaining points")

format_axes(axes[0], "Line refinement")
format_axes(axes[1], "After removing line inliers")

for ax in axes:
    ax.legend()

plt.tight_layout()
plt.show()

a, b, d = line_best
print(f"Refined line: {a:.4f}x + {b:.4f}y = {d:.4f}")
print(f"Normal magnitude: {np.hypot(a, b):.4f}")
print(f"Final line inliers: {line_mask.sum()}")
print(f"Remaining points: {len(remaining)}")
```



```
Refined line: 0.7179x + 0.6962y = 1.5209
Normal magnitude: 1.0000
Final line inliers: 58
Remaining points: 42
```

In [21]:

```python
def fit_circle_sample(points):
    p1, p2, p3 = points

    A = 2 * np.vstack((p2 - p1, p3 - p1))
    B = np.array([
        p2 @ p2 - p1 @ p1,
        p3 @ p3 - p1 @ p1
    ])

    if np.linalg.cond(A) > 1e10:
        raise ValueError("Collinear or nearly collinear points.")

    center = np.linalg.solve(A, B)
    radius = np.linalg.norm(p1 - center)

    return np.array([center[0], center[1], radius])


def circle_error(model, points):
    return np.linalg.norm(points - model[:2], axis=1) - model[2]


circle_threshold = 1.5
min_circle_consensus = 25

best_score = (-1, -np.inf)
circle_sample_model = None

if len(remaining) < 3:
    raise RuntimeError("Insufficient remaining points.")

for _ in range(iterations):
    sample = remaining[rng.choice(len(remaining), 3, replace=False)]

    try:
        model = fit_circle_sample(sample)
    except (ValueError, np.linalg.LinAlgError):
        continue

    errors = np.abs(circle_error(model, remaining))
    mask = errors < circle_threshold

    score = (int(mask.sum()), -float(np.sum(errors[mask]**2)))

    if score > best_score:
        best_score = score
        circle_sample_model = model.copy()
```

```
        circle_sample_points = sample.copy()
        circle_sample_mask = mask.copy()

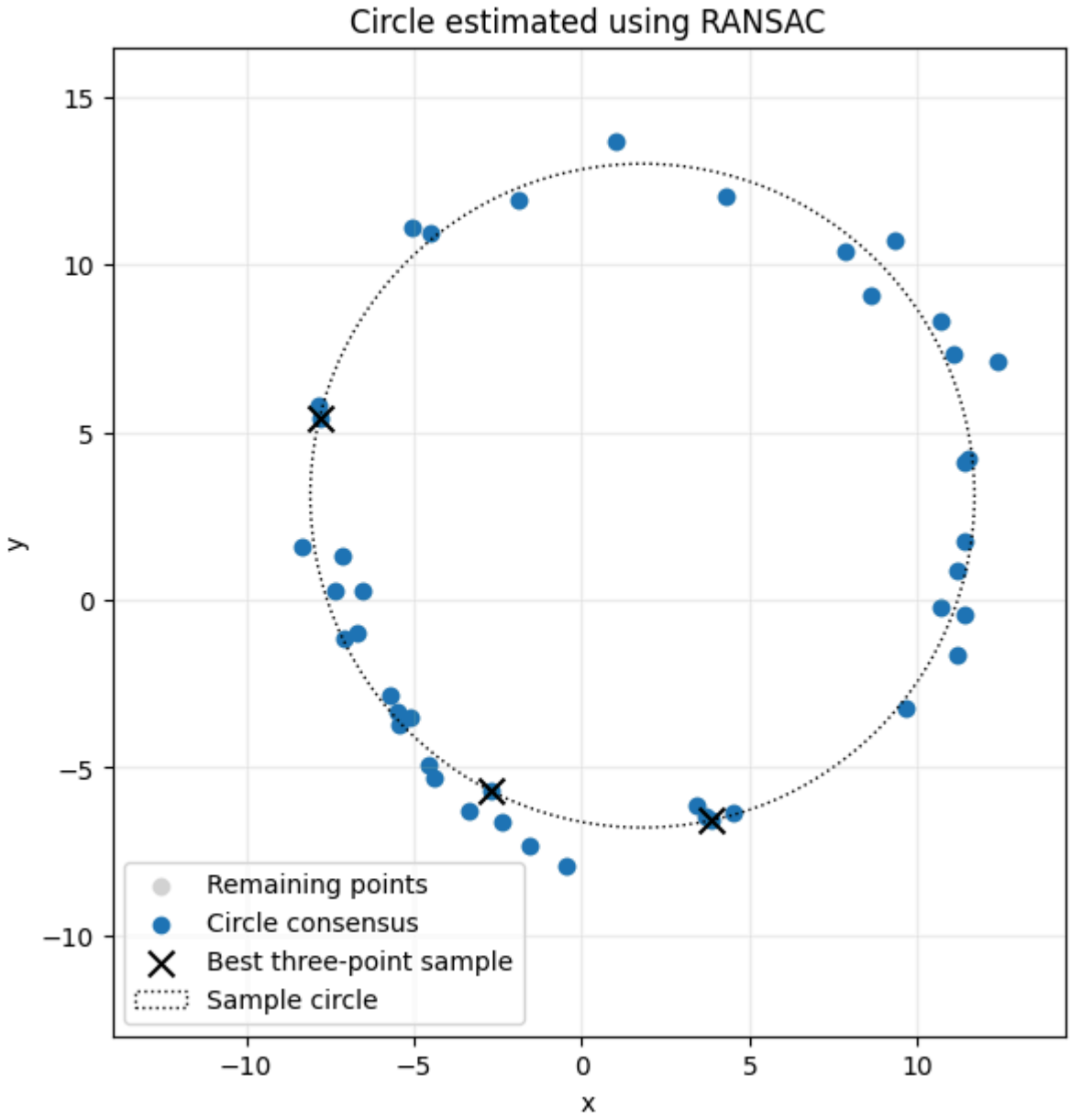
if circle_sample_model is None or best_score[0] < min_circle_consensus:
    raise RuntimeError("No circle met the minimum consensus requirement.")

fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(*remaining.T, color="lightgray", label="Remaining points")
ax.scatter(
    *remaining[circle_sample_mask].T,
    color="tab:blue", label="Circle consensus"
)
ax.scatter(
    *circle_sample_points.T, color="black", marker="x", s=100,
    label="Best three-point sample", zorder=5
)

ax.add_patch(
    plt.Circle(
        circle_sample_model[:2], circle_sample_model[2],
        fill=False, color="black", linestyle=":",
        label="Sample circle"
    )
)

format_axes(ax, "Circle estimated using RANSAC")
ax.legend()
plt.show()

cx, cy, radius = circle_sample_model
print(f"Sample circle centre: ({cx:.3f}, {cy:.3f})")
print(f"Sample radius: {radius:.3f}")
print(f"Consensus size: {circle_sample_mask.sum()}")
```




```
Sample circle centre: (1.809, 3.113)
Sample radius: 9.898
Consensus size: 42
```

In [22]:

```
result = minimize(
    lambda model: np.sum(
        circle_error(model, remaining[circle_sample_mask])**2
    ),
    x0=circle_sample_model,
    method="L-BFGS-B",
    bounds=[(None, None), (None, None), (1e-6, None)]
)

if not result.success:
    raise RuntimeError(f"Circle refinement failed: {result.message}")

circle_best = result.x

circle_mask = (
```

```
    np.abs(circle_error(circle_best, remaining)) < circle_threshold
)

if circle_mask.sum() < min_circle_consensus:
    raise RuntimeError("Refined circle has insufficient consensus.")

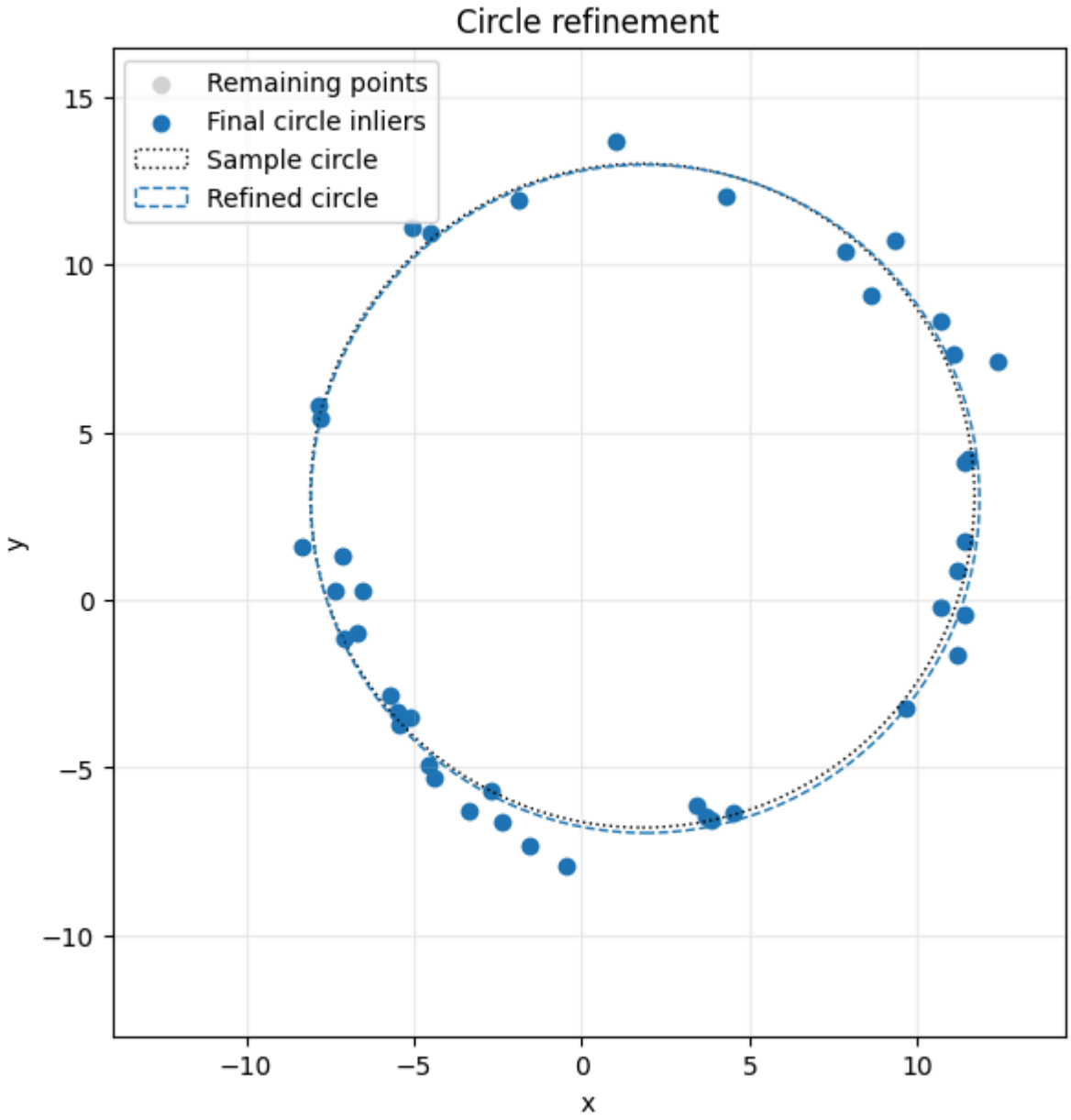
# Compare errors on the same consensus used for refinement.
fit_points = remaining[circle_sample_mask]
rmse_before = np.sqrt(np.mean(circle_error(circle_sample_model,
fit_points)**2))
rmse_after = np.sqrt(np.mean(circle_error(circle_best, fit_points)**2))

fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(*remaining.T, color="lightgray", label="Remaining points")
ax.scatter(
    *remaining[circle_mask].T,
    color="tab:blue", label="Final circle inliers"
)

for model, color, style, label in [
    (circle_sample_model, "black", ":", "Sample circle"),
    (circle_best, "tab:blue", "--", "Refined circle")
]:
    ax.add_patch(
        plt.Circle(
            model[:2], model[2], fill=False,
            color=color, linestyle=style, label=label
        )
    )

format_axes(ax, "Circle refinement")
ax.legend()
plt.show()

cx, cy, radius = circle_best
print(f"Refined centre: ({cx:.3f}, {cy:.3f})")
print(f"Refined radius: {radius:.3f}")
print(f"Final circle inliers: {circle_mask.sum()}")
print(f"Consensus radial RMSE: {rmse_before:.4f} → {rmse_after:.4f}")
```




```
Refined centre: (1.893, 3.018)
Refined radius: 9.964
Final circle inliers: 42
Consensus radial RMSE: 0.5723 → 0.5615
```

In [23]:

```
fig, ax = plt.subplots(figsize=(9, 9))

ax.scatter(*X.T, color="lightgray", s=25, label="All points")
ax.scatter(
    *X[line_mask].T, color="orange", s=30,
    label=f"Line inliers ({line_mask.sum()})"
)
ax.scatter(
    *remaining[circle_mask].T, color="tab:blue", s=30,
    label=f"Circle inliers ({circle_mask.sum()})"
)
ax.scatter(
    *remaining[~circle_mask].T, color="gray", s=30,
    label="Remaining outliers"
```

```
)

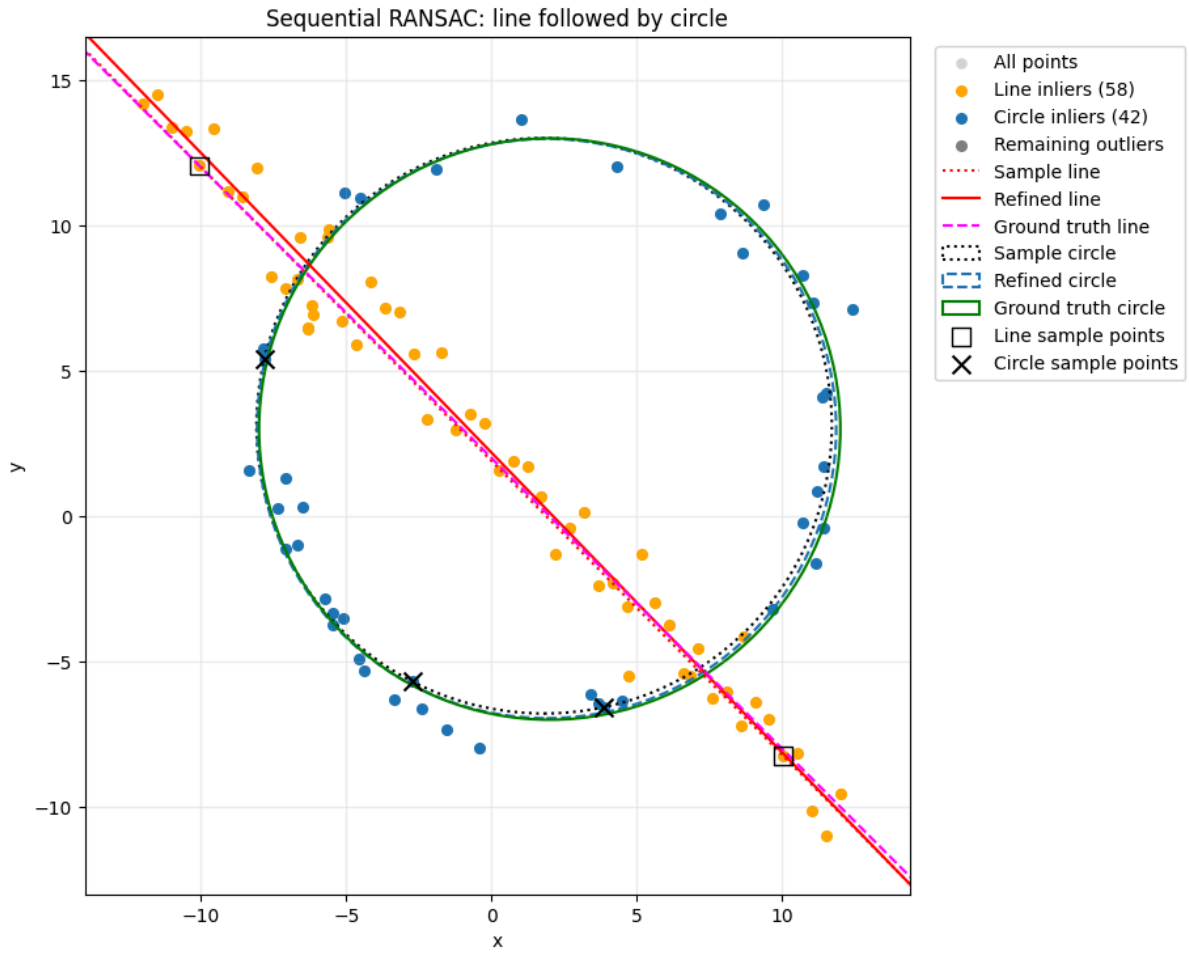
draw_line(ax, line_sample_model, "red", ":", "Sample line")
draw_line(ax, line_best, "red", "-", "Refined line")
draw_line(
    ax, np.array([1, 1, 2]) / np.sqrt(2),
    "magenta", "--", "Ground truth line"
)

for model, color, style, label in [
    (circle_sample_model, "black", ":", "Sample circle"),
    (circle_best, "tab:blue", "--", "Refined circle"),
    (np.array([2, 3, 10]), "green", "-", "Ground truth circle")
]:
    ax.add_patch(
        plt.Circle(
            model[:2], model[2], fill=False,
            edgecolor=color, linestyle=style,
            linewidth=1.5, label=label
        )
    )

ax.scatter(
    *line_sample_points.T,
    marker="s", s=100, facecolors="none", edgecolors="black",
    label="Line sample points", zorder=5
)
ax.scatter(
    *circle_sample_points.T,
    marker="x", s=100, color="black",
    label="Circle sample points", zorder=5
)

format_axes(ax, "Sequential RANSAC: line followed by circle")
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1))

plt.tight_layout()
plt.show()
```



What happens if the circle is fitted first?

Fitting the circle first can succeed, but it may select an incorrect, very large circle whose arc approximates the line. Such a circle can collect many line points as inliers.

Removing the line first reduces this ambiguity. However, some circle points near the line–circle intersections may also be removed, leaving fewer points for circle estimation.

Question 3: Image Superimposition Using Homography

In [42]:

```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
alpha=[0.5,0.8,0,3]
bgs= [ "wadham.jpg",  "roof.jpg", "defender.jpg"]
overlays =["flag.png","entc logo.jpg","Bajaj.png"]
for i in range(3):


    # 1. Load Images
    folder = "C:/Users/ASUS/Downloads/CV ASSIGNMENT/A02-on-Fitting-and-
Alignment/a2images/"
    background = cv.imread(folder + bgs[i])
    overlay = cv.imread(folder +overlays[i])

    if background is None or overlay is None:
        raise FileNotFoundError("Check the image paths.")
```

```
    # Display 1: Initial Background and Overlay
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    for ax, image, title in zip(axes, [background, overlay], ["Background",
"Overlay"]):
        ax.imshow(cv.cvtColor(image, cv.COLOR_BGR2RGB))
        ax.set_title(title)
        ax.axis("off")
    plt.tight_layout()
    plt.show()

    # 2. Interactive Corner Selection Function
    def select_corners(image):
        scale = min(1.0, 1000 / image.shape[1], 700 / image.shape[0])
        preview = cv.resize(image, None, fx=scale, fy=scale)
        sx = preview.shape[1] / image.shape[1]
        sy = preview.shape[0] / image.shape[0]

        points = []
        window = "Select corners: TL, TR, BR, BL (Press ENTER when done, 'r'
to reset)"

        def click(event, x, y, flags, param):
            if event == cv.EVENT_LBUTTONDOWN and len(points) < 4:
                points.append((x, y))

        cv.namedWindow(window)
        cv.setMouseCallback(window, click)

        try:
            while True:
                display = preview.copy()
                for number, point in enumerate(points, 1):
                    cv.circle(display, point, 5, (0, 0, 255), -1)
                    cv.putText(display, str(number), (point[0] + 8, point[1]
- 8),
                               cv.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
                if len(points) > 1:
                    cv.polylines(display, [np.array(points,
dtype=np.int32)], len(points) == 4, (0, 255, 0), 2)

                cv.imshow(window, display)
                key = cv.waitKey(20) & 0xFF

                if key in (10, 13) and len(points) == 4:
                    break
                elif key in (ord("r"), ord("R")):
                    points.clear()
                elif key == 27:
                    raise RuntimeError("Corner selection cancelled.")
                if cv.getWindowProperty(window, cv.WND_PROP_VISIBLE) < 1:
                    raise RuntimeError("Selection window closed.")
```

```
        finally:
            cv.destroyAllWindows()

        return np.array(points, dtype=np.float32) / np.array([sx, sy],
dtype=np.float32)

    # 3. Get Target Quadrilateral Corners
    print("Please select 4 corners on the background image in order: Top-
Left, Top-Right, Bottom-Right, Bottom-Left.")
    corners = select_corners(background)

    contour = corners.reshape(-1, 1, 2)
    if not cv.isContourConvex(contour) or cv.contourArea(contour) < 1:
        raise ValueError("Select four corners of a convex region in the
stated order.")

    # Display 2: Selected Target Region with Polygon & Corner Numbers
    plt.figure(figsize=(8, 6))
    plt.imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB))
    closed = np.vstack((corners, corners[0]))
    plt.plot(closed[:, 0], closed[:, 1], "r-o")
    for number, (x, y) in enumerate(corners, 1):
        plt.text(x + 5, y, str(number), color="yellow", fontsize=12,
fontweight='bold')
    plt.title("Selected target region")
    plt.axis("off")
    plt.show()

    print("Selected corners:\n", corners)

    # 4. Compute Homography and Warp Overlay
    h, w = overlay.shape[:2]
    height, width = background.shape[:2]

    source_corners = np.float32([
        [0, 0],
        [w - 1, 0],
        [w - 1, h - 1],
        [0, h - 1]
    ])

    H, _ = cv.findHomography(source_corners, corners, method=0)
    if H is None or not np.isfinite(H).all():
        raise RuntimeError("Homography estimation failed. Reselect the
corners.")

    warped = cv.warpPerspective(overlay, H, (width, height))
    mask = cv.warpPerspective(np.full((h, w), 255, dtype=np.uint8), H,
(width, height), flags=cv.INTER_NEAREST)

    # Display 3: Warped Overlay & Overlay Mask
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
    axes[0].imshow(cv.cvtColor(warped, cv.COLOR_BGR2RGB))
    axes[0].set_title("Warped overlay")

    axes[1].imshow(mask, cmap="gray")
    axes[1].set_title("Overlay mask")

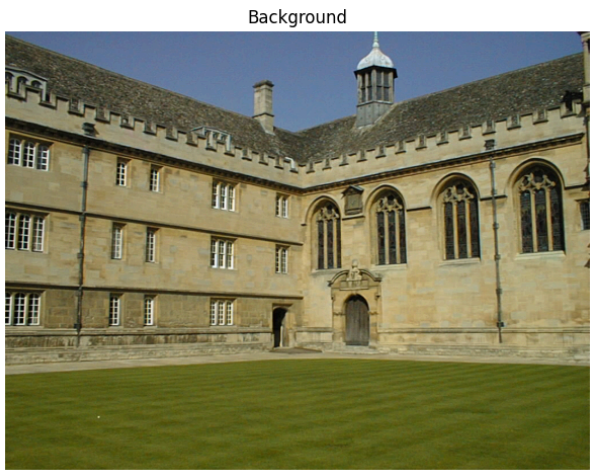
    for ax in axes:
        ax.axis("off")
    plt.tight_layout()
    plt.show()

    print("Homography matrix:\n", H)

    # 5. Blend Overlay onto Background
    alpha = 0.8
    mixed = cv.addWeighted(background, 1 - alpha, warped, alpha, 0)

    blended = background.copy()
    blended[mask > 0] = mixed[mask > 0]

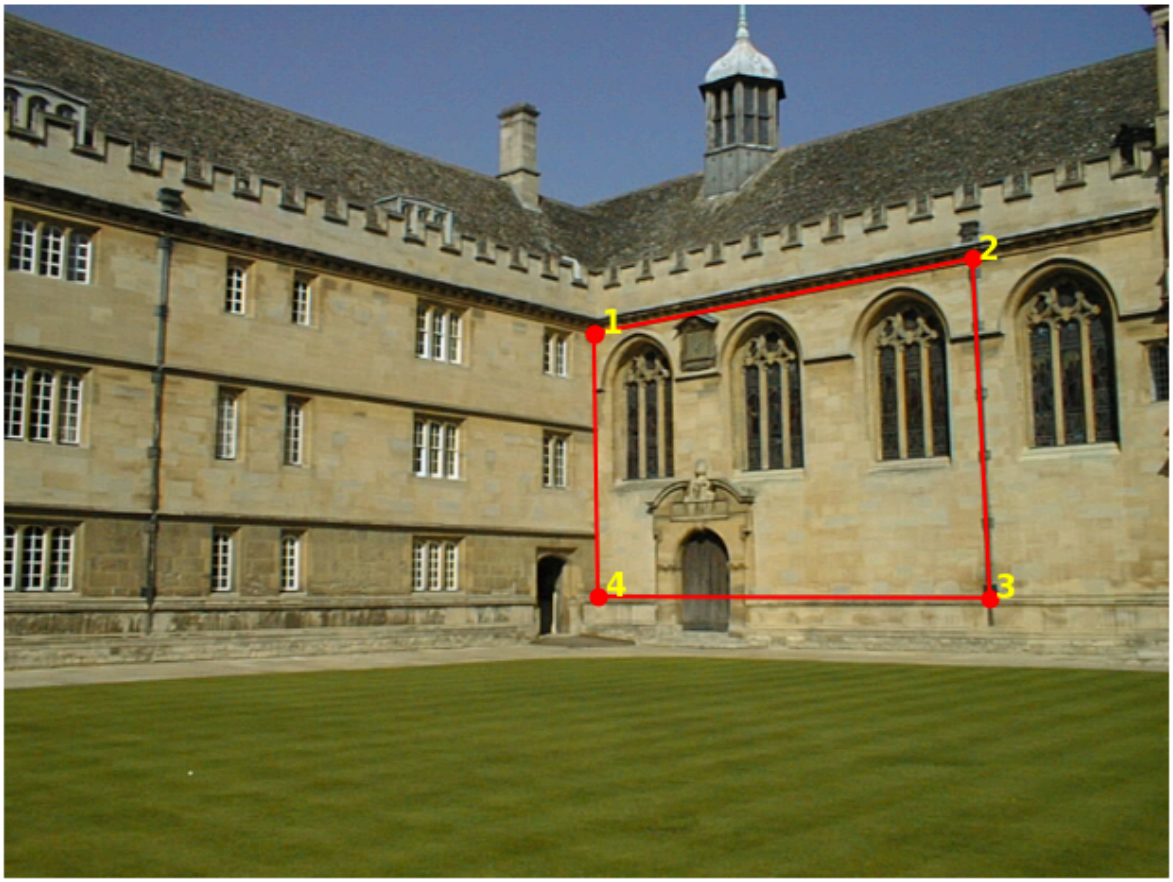
    # Display 4: Final Comparison Summary (Background, Overlay, Blended
Result)
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    for ax, image, title in zip(
        axes,
        [background, overlay, blended],
        ["Background", "Overlay", "Blended result"]
    ):
        ax.imshow(cv.cvtColor(image, cv.COLOR_BGR2RGB))
        ax.set_title(title)
        ax.axis("off")
    plt.tight_layout()
    plt.show()
```






```
Please select 4 corners on the background image in order: Top-Left, Top-
Right, Bottom-Right, Bottom-Left.
```

Selected target region

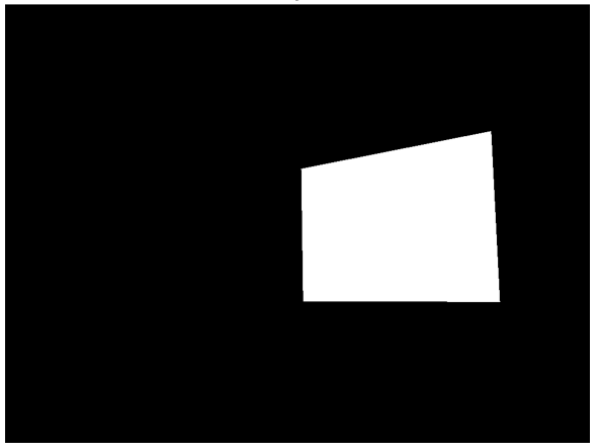



```
Selected corners:
 [[518.03644 288.54858]
 [850.5895  222.72   ]
 [865.95496 521.1429 ]
 [521.32904 520.0457 ]]
```

Warped overlay

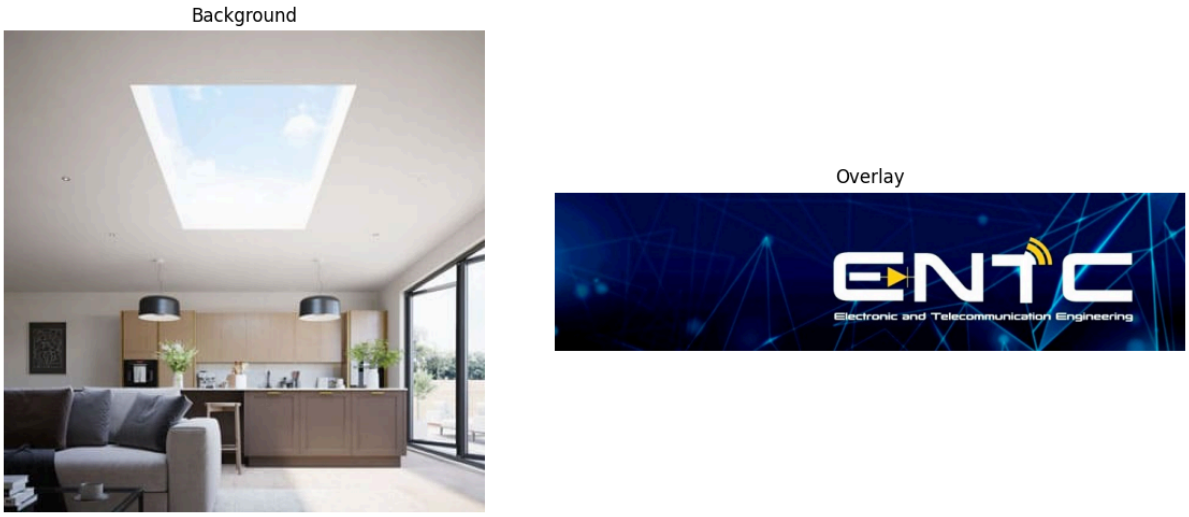


Overlay mask

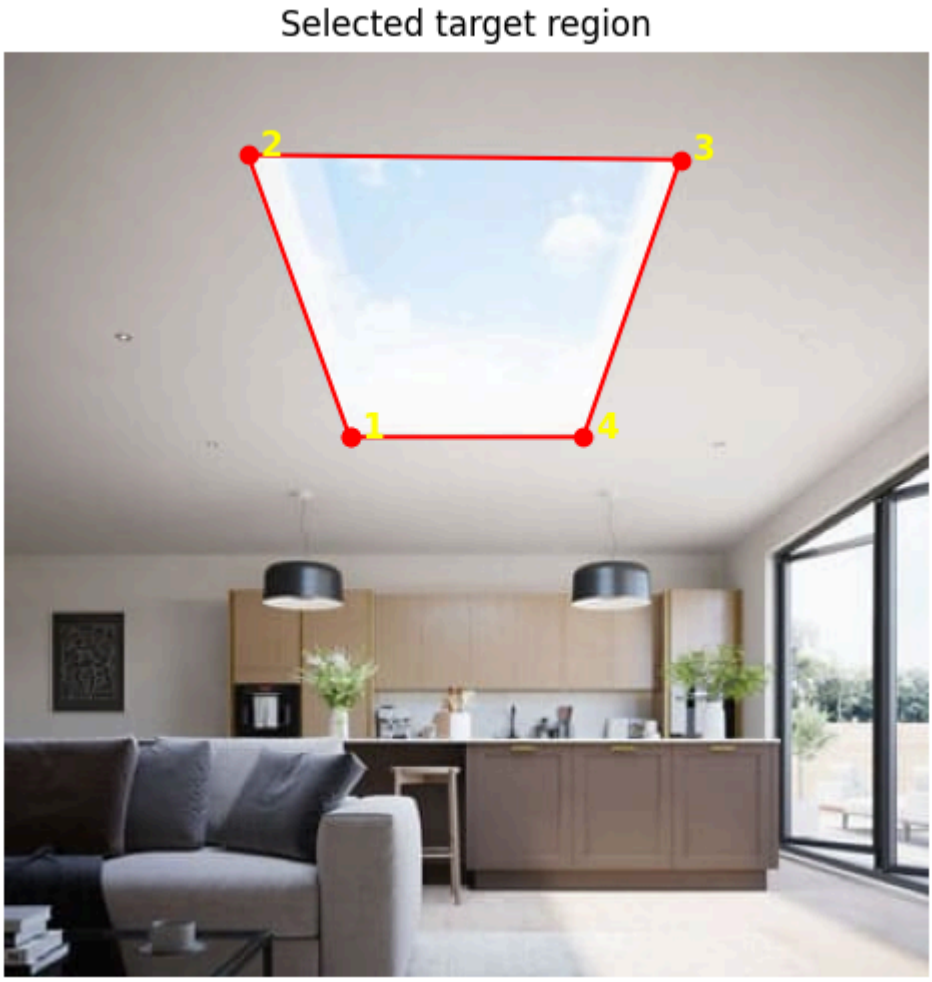


```
Homography matrix:
 [[ 7.39312396e-02 -1.01771062e-02  5.18036438e+02]
 [-6.03211778e-02  2.27817325e-01  2.88548584e+02]
 [-1.16817488e-04 -2.61072680e-05  1.00000000e+00]]
```



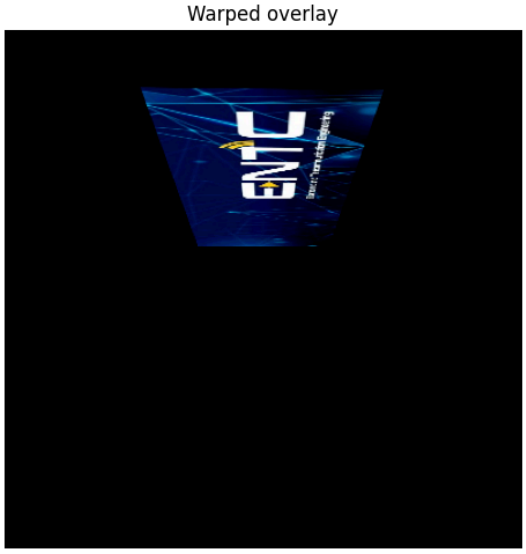



```
Please select 4 corners on the background image in order: Top-Left, Top-
Right, Bottom-Right, Bottom-Left.
```

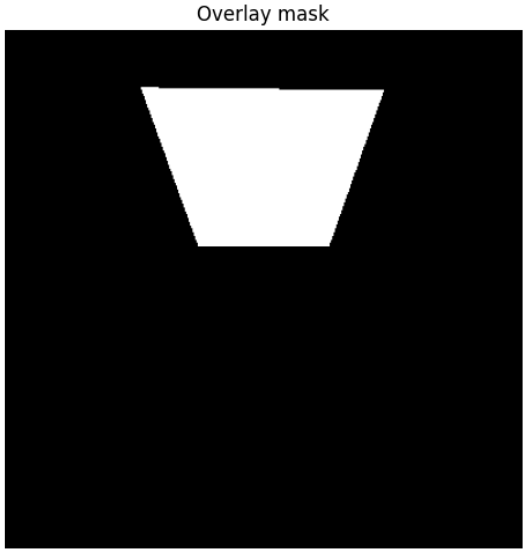


```
Selected corners:
 [[149. 166.]
 [105.  44.]
 [292.  46.]
 [250. 166.]]
```

Warped overlay



Overlay mask



```
Homography matrix:
 [[-1.15770046e-01  5.18804286e-01  1.49000000e+02]
 [-1.78127561e-01  7.48102080e-03  1.66000000e+02]
 [-5.78106757e-04  4.50663904e-05  1.00000000e+00]]
```

Background



Overlay



Blended result



Background



Overlay




```
Please select 4 corners on the background image in order: Top-Left, Top-
Right, Bottom-Right, Bottom-Left.
```

Selected target region

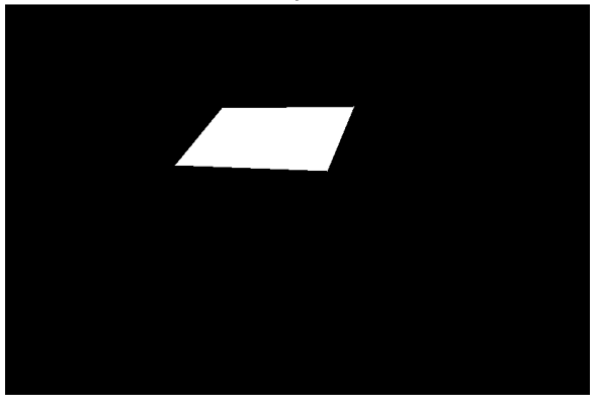



```
Selected corners:
 [[300. 143.]
 [483. 141.]
 [446. 230.]
 [235. 222.]]
```

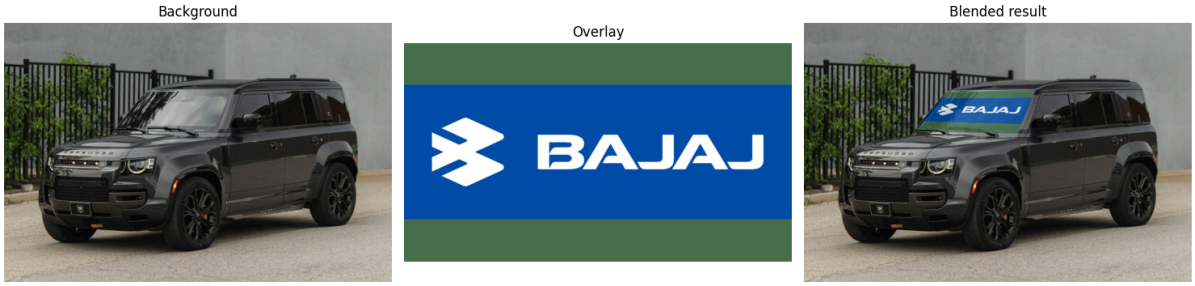
Warped overlay



Overlay mask



```
Homography matrix:
 [[ 4.22770877e-02 -5.56995777e-02  3.00000000e+02]
 [-4.98314308e-03  2.53840396e-02  1.43000000e+02]
 [-3.09074305e-05 -8.33551769e-05  1.00000000e+00]]
```



Question 4: PCB Image Alignment and Difference Analysis

In [45]:

```
from pathlib import Path

folder = Path("C:/Users/ASUS/Downloads/CV ASSIGNMENT/A02-on-Fitting-and-Alignment/a2images/pcbs")

template = cv.imread(str(folder / "AECC1.jpg"), cv.IMREAD_GRAYSCALE)
h, w = template.shape

sift = cv.SIFT_create()
matcher = cv.BFMatcher(cv.NORM_L2)

kp_ref, des_ref = sift.detectAndCompute(template, None)

print(f"Reference keypoints: {len(kp_ref)}")

image_names = ["AECC2.jpg", "AAAA1.jpg", "ADAD1.jpg", "AECC3.jpg", "AECC4.jpg"]

for name in image_names:
    image = cv.imread(str(folder / name), cv.IMREAD_GRAYSCALE)
    kp, des = sift.detectAndCompute(image, None)

    # Match descriptors and apply the ratio test.
    pairs = matcher.knnMatch(des, des_ref, k=2)
    good_matches = [
        m for m, n in pairs if m.distance < 0.75 * n.distance
    ]

    # Keep the closest match for each reference keypoint.
    best_matches = {}

    for match in good_matches:
        previous = best_matches.get(match.trainIdx)

        if previous is None or match.distance < previous.distance:
            best_matches[match.trainIdx] = match

    good_matches = list(best_matches.values())

    # Estimate the homography: input image → reference.
    src = np.float32([
        kp[m.queryIdx].pt for m in good_matches
```

```
    ]).reshape(-1, 1, 2)

    dst = np.float32([
        kp_ref[m.trainIdx].pt for m in good_matches
    ]).reshape(-1, 1, 2)

    H, inlier_mask = cv.findHomography(src, dst, cv.RANSAC, 5.0)

    aligned = cv.warpPerspective(image, H, (w, h))

    # Identify valid overlapping pixels.
    coverage = cv.warpPerspective(
        np.ones(image.shape, dtype=np.float32), H, (w, h)
    )
    valid = coverage > 0.999

    difference = cv.absdiff(template, aligned)
    mae = difference[valid].mean()

    # Display up to 80 RANSAC inlier matches.
    inlier_matches = [
        m for m, keep in zip(good_matches, inlier_mask.ravel()) if keep
    ]
    inlier_matches.sort(key=lambda m: m.distance)

    match_image = cv.drawMatches(
        image, kp,
        template, kp_ref,
        inlier_matches[:80], None,
        matchColor=(0, 255, 0),
        flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(14, 6))
    plt.imshow(cv.cvtColor(match_image, cv.COLOR_BGR2RGB))
    plt.title(f"{name} → AECC1.jpg: inlier matches (up to 80 shown)")
    plt.axis("off")
    plt.tight_layout()
    plt.show()

    # Display aligned image and absolute difference.
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    images = [
        template,
        aligned,
        np.ma.array(difference, mask=~valid)
    ]
    titles = ["Reference", f"Aligned: {name}", "Absolute difference"]

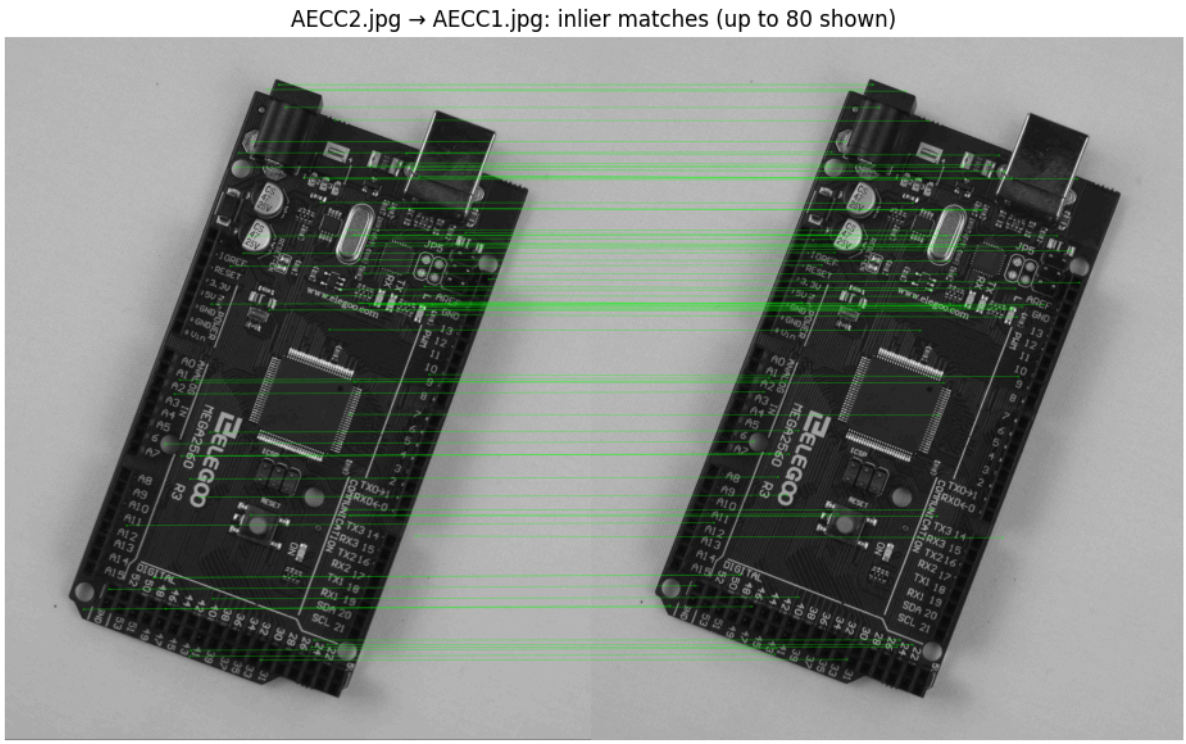
    for ax, output, title in zip(axes, images, titles):
        ax.imshow(output, cmap="gray", vmin=0, vmax=255)
```

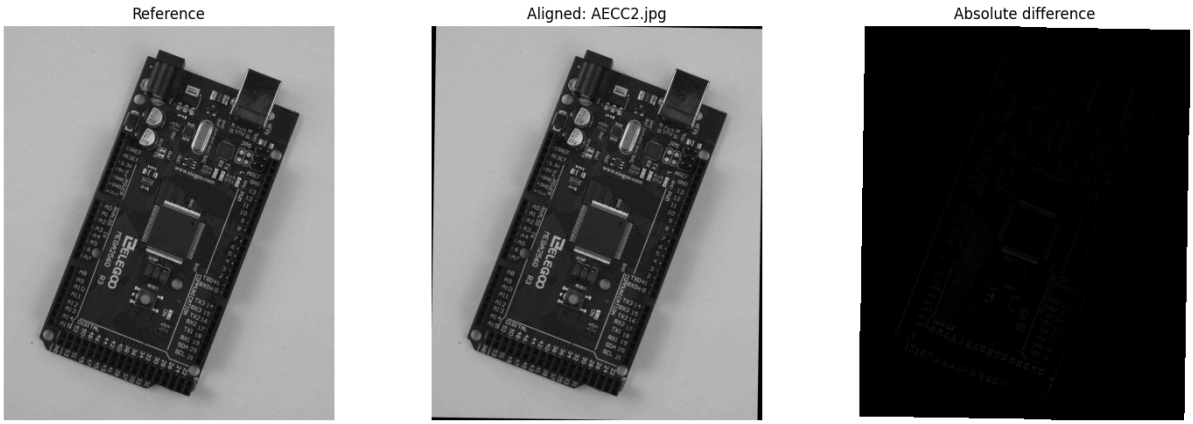
```
        ax.set_title(title)
        ax.axis("off")

    plt.tight_layout()
    plt.show()

    print(
        f"{name}: keypoints={len(kp)}, "
        f"unique matches={len(good_matches)}, "
        f"inliers={len(inlier_matches)}, "
        f"overlap={100 * valid.mean():.1f}%, "
        f"MAE={mae:.2f}"
    )
```

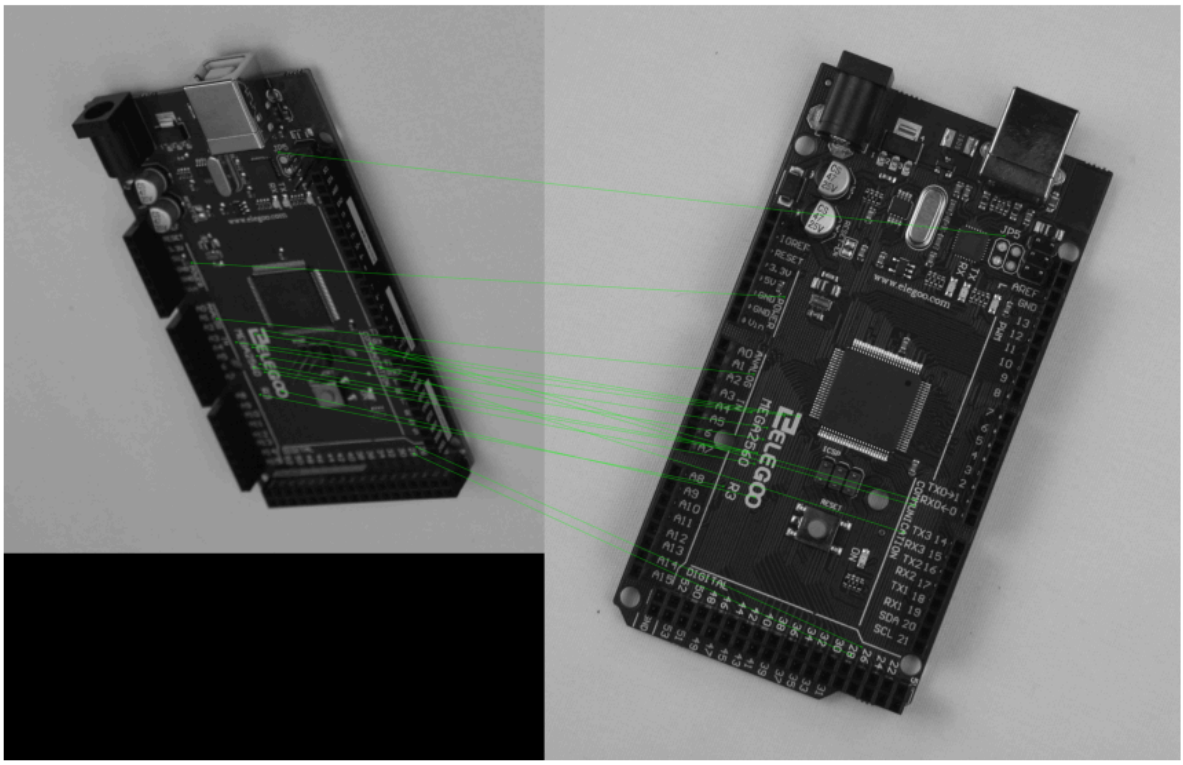
```
Reference keypoints: 14271
```



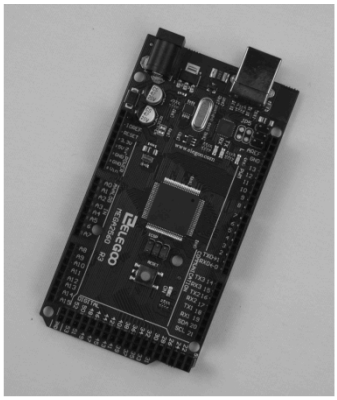



```
AECC2.jpg: keypoints=12312, unique matches=4110, inliers=3744,
overlap=98.4%, MAE=3.00
```

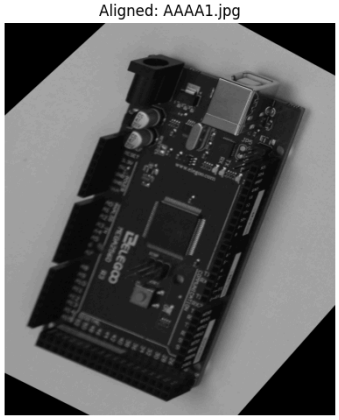
AAAA1.jpg → AECC1.jpg: inlier matches (up to 80 shown)



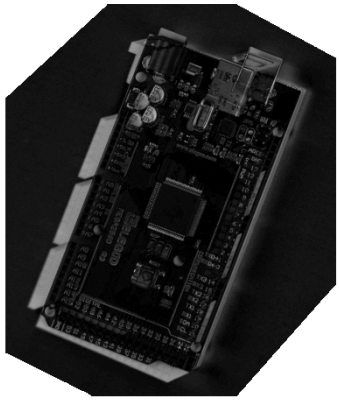
Reference



Aligned: AAAA1.jpg

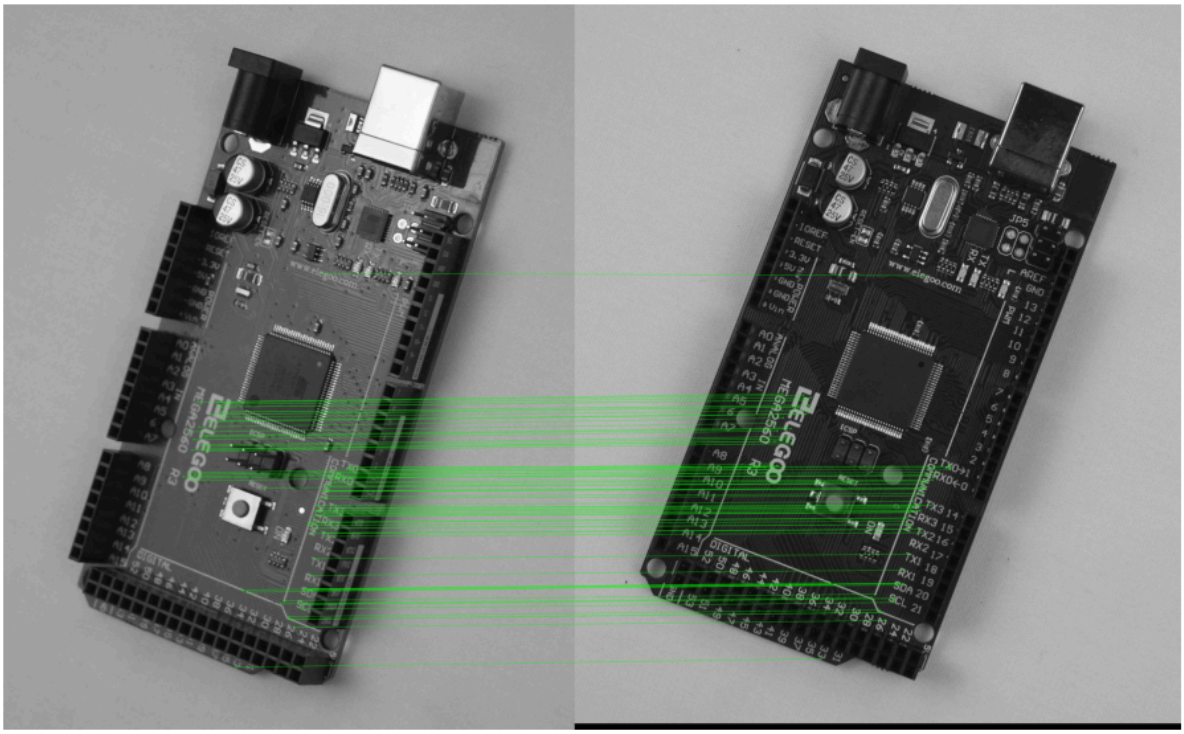


Absolute difference

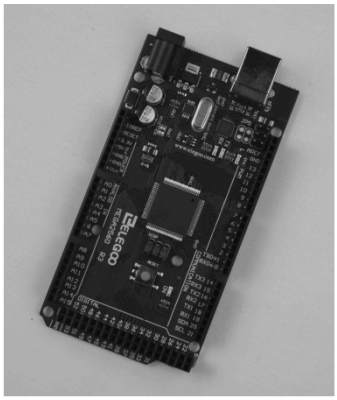


AAAA1.jpg: keypoints=2413, unique matches=47, inliers=18, overlap=88.8%, MAE=28.32

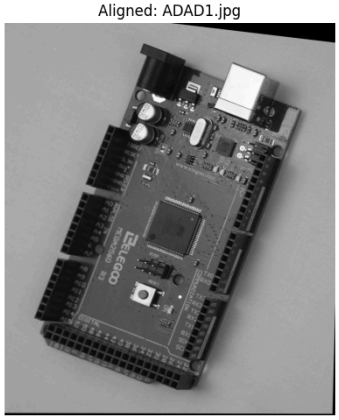
ADAD1.jpg → AECC1.jpg: inlier matches (up to 80 shown)



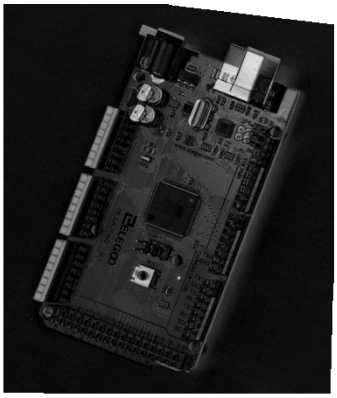
Reference



Aligned: ADAD1.jpg

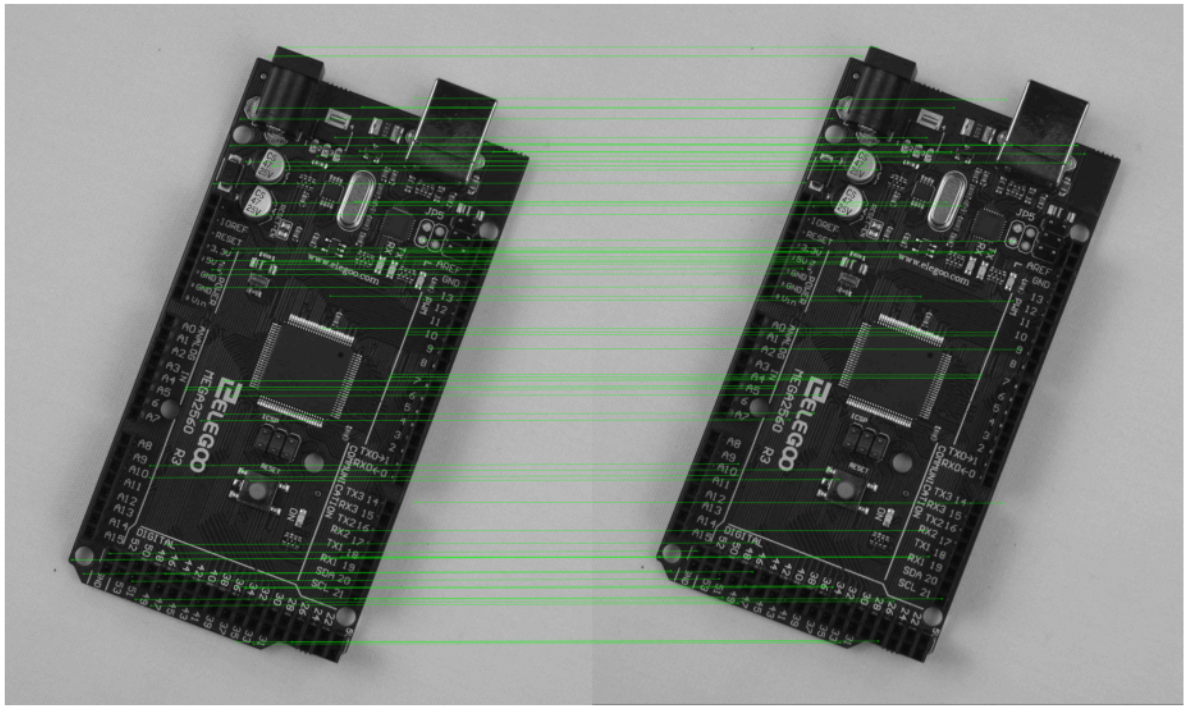


Absolute difference

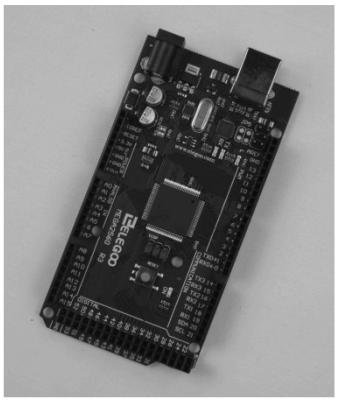


ADAD1.jpg: keypoints=8488, unique matches=300, inliers=190, overlap=98.1%, MAE=31.15

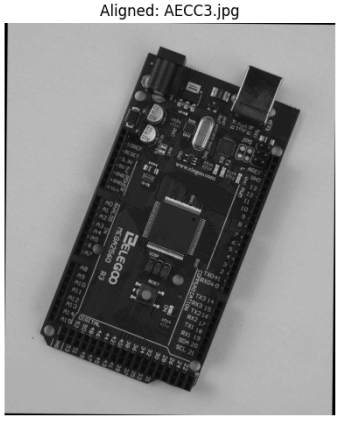
AECC3.jpg → AECC1.jpg: inlier matches (up to 80 shown)



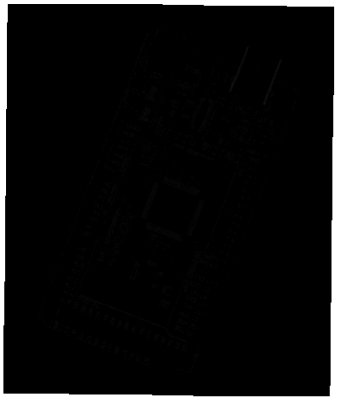
Reference



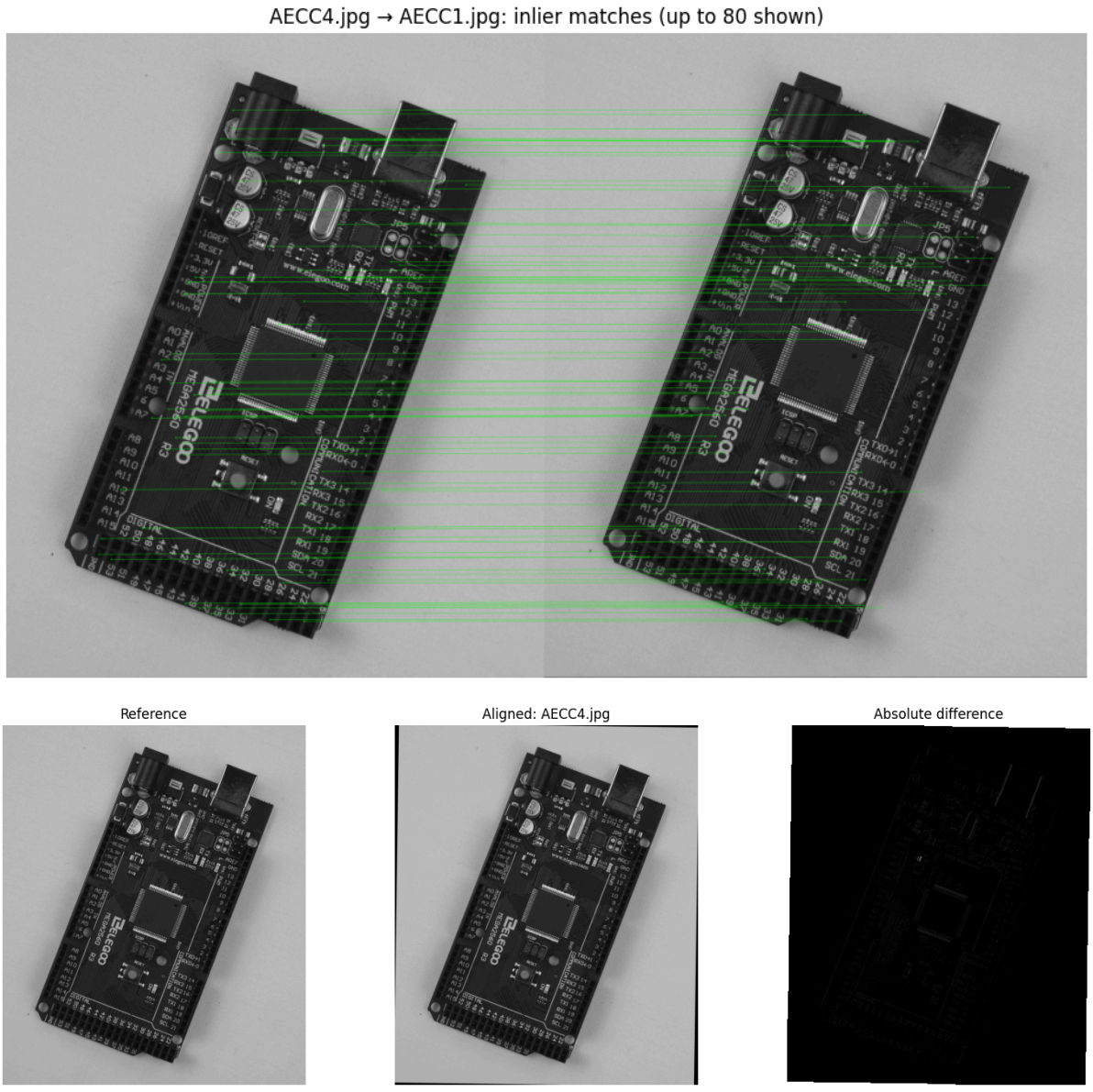
Aligned: AECC3.jpg



Absolute difference



AECC3.jpg: keypoints=15197, unique matches=5332, inliers=4849, overlap=98.9%, MAE=2.78




```
AECC4.jpg: keypoints=11610, unique matches=3602, inliers=3288,
overlap=98.5%, MAE=3.58
```

PCB Alignment Using SIFT and RANSAC

SIFT features were extracted from each PCB image and the reference, **AECC1.jpg**. Features were matched using a brute-force matcher. A ratio test of **0.75** rejected ambiguous matches, and duplicate matches to reference keypoints were removed.

RANSAC estimated a homography using a **5-pixel** reprojection threshold. Each image was warped to align with the reference, and the absolute grayscale difference was calculated:

$$D = |I_{\text{reference}} - I_{\text{aligned}}|$$

The matches, aligned images, and difference images were displayed. MAE was calculated only over valid overlapping pixels.

Why do raised components show high differences?

The homography aligns the planar PCB surface. Raised components, such as the USB port, capacitors, and reset button, experience **parallax** and may remain misaligned. Shadows and reflections also change between views, so high differences do not necessarily indicate defects.